

Pintos Project 2: User Program (2)

담당 교수 / 분반 : 김영재

이름 / 학번 : 방지혁 / 20190808

개발 기간 : 2025.10.03 ~ 2025.10.28

I. 개발 목표

이전까지는 기본적인 시스템 콜 구현, 프로세스 간 hierarchy 구현을 했었고, 이번 프로젝트에서의 주 목표는 사용자 프로그램이 파일을 생성하거나, 읽거나, 열거나, 쓰거나 혹은 닫을 때 이를 커널에서 지원하도록 관련 시스템콜들을 구현합니다. 또한, 이에서 발행할 수 있는 동기화 문제도 해결합니다.

II. 개발 범위 및 내용

A. 개발 범위

1. File Descriptor

Project 1까지의 구현상 임의의 프로세스가 파일을 읽어오는 것도 구현이 안되었을 뿐더러 개별적인 파일을 읽어왔다고 하더라도 이를 저장하고 식별하여 접근할 방법이 없었습니다. 이를 위해 각 프로세스가 독립적인 파일 디스크립터 테이블, 즉 배열을 선언합니다. 이를 통해서 STDIN, STDOUT, STDERR 뿐만 아니라 다른 파일에 접근할 수 있게 됩니다.

2. System Calls

Project 1에서 구현한 read와 write의 경우 STDIN, STDOUT에 대해서만 가능하도록 설계했기 때문에 이를 해결하고 새로운 system call로는 파일 크기를 반환하는 filesize, 파일 내 위치를 이동하는 seek, 현재 파일 내 위치를 반환하는 tell, 새로운 파일을 생성하는 create, 파일을 삭제하는 remove, 파일을 여는 open, 파일을 닫는 close를 구현했습니다.

3. Synchronization in Filesystem

여러 프로세스가 같은 파일에 접근하는 경우가 발생할 수 있습니다. 예를 들자면 어떤 프로세스가 파일을 쓰고 있을 때, 다른 프로세스에서 해당 파일을 접근한다거나, 어떤 프로세스의 실행 파일 자체를 수정하려고 하는 시도가 있을 수 있습니다. 이는 무결성을 위반하는 것으로 이를 위해 lock을 사용하거나 세마포어를 사용해야 합니다.

B. 개발 내용

1. File Descriptor

File descriptor table 구현을 위해 배열을 선택했습니다. 별도의 구조체를 선언하지 않고

file 포인터를 직접 배열에 저장하도록 했습니다. 가장 다루기 쉬운 자료형이며, 한 프로세스가 열 파일의 개수는 제한적이라고 생각했기 때문에 고정 크기 배열로도 충분할 것이라고 판단했기 때문입니다.

2. System Calls

create: 기존의 `filesys_create()` 함수를 호출하여 구현했습니다. 그리고 해당 이름과 크기를 가진 파일을 생성하는데 만약 성공 시 `true`, 실패 시 `false`를 반환한다.

Remove: 기존의 `filesys_remove()` 함수를 호출하여 구현했습니다. 마찬가지로 file 이름을 받아 성공 시 `true`, 실패 시 `false`를 반환합니다.

Read: 일반 파일의 경우 `file_read()` 함수를 호출하도록 구현했습니다. 실제로 읽은 bytes 수를 반환합니다.

Write: 일반 파일의 경우 `file_write()` 함수를 호출하도록 구현했습니다. Read와 비슷하게 실제로 쓴 bytes 수를 반환합니다.

Open: `filesys_open()` 함수를 호출하여 구현하고 사용 가능한 가장 작은 fd를 할당하여 해당 테이블에 포인터를 저장하고 fd 값을 반환합니다.

Close: 우선 입력 받은 fd의 유효성을 확인하고 `file_close` 함수로 해당 파일을 닫습니다.

Seek: fd값을 받아 유효성을 확인하고 `file_seek()` 함수를 호출하여 파일 포인터를 해당 위치로 이동시킵니다.

Tell: fd값을 받아 유효성을 확인하고 `file_tell()` 함수를 호출하여 파일 내 포인터의 현재 위치를 반환합니다.

Filesize: fd 값을 받아 유효성을 확인하고 `file_length()` 함수를 호출하여 파일의 bytes 크기를 반환합니다.

3. Synchronization in Filesystem

우선 해당 프로젝트에서 구현한 lock 사용방법에 대해 설명하겠습니다. 전역 락인 `Struct lock filesys_lock`을 선언하여 파일 접근 함수를 호출하기 전 시스템 콜의 앞부분에서 `lock_acquire(&filesys_lock)`을 통해 lock을 획득하고, 완료되면 `lock_release(&filesys_lock)`을 통해 해제해야 합니다. 한편 semaphore를 사용한다면 우리가 시스템 프로그래밍 수업에서 배웠듯이 reader-writer 패턴을 사용하는 것입니다. 파일별로 Semaphore를 통해 동시 읽기에 대해서는 허용하지만, 쓰기에 대해서는 배타적으로만 허용하는 것입니다. 구현방법

에 대해 설명하자면 각 파일마다 `sema_init(&file_sema, 1)`로 초기화하고, 각 프로세스는 `sema_down(&file_sema)`로 파일 접근 전 세마포어를 낮추고, 작업이 완료되면 `sema_up(&file_sema)`로 세마포어를 올려 다른 프로세스에게 접근이 가능하다는 것을 알려주는 것입니다. Lock이 더 간단하지만 성능적인 측면에서는 세마포어가 낫습니다.

III. 추진 일정 및 개발 방법

A. 추진 일정

10/03 ~ 10/10: file descriptor 구조 설계

10/11 ~ 10/18: 파일 system call 구현

10/19 ~ 10/26: 디버깅 및 문제 해결

10/27 ~ 10/28: 보고서 작성

B. 개발 방법

1. File Descriptor

threads/thread.h 수정

file descriptor table 및 실행 파일 포인터 추가

```
struct file *fdt[128]; struct file *current_file;
```

threads/thread.c 수정

`init_thread()` 함수에서 file descriptor table 초기화

userprog/process.c 수정

`process_exit()` 함수에서 열린 파일 닫기, `load()` 함수에서 파일 포인터 이용하여 실행 파일에 대해 write 금지

2. System Calls

userprog/syscall.c

`syscall_handler`에 추가로 구현된 시스템 콜들에 대해 case문 추가 및 추가 시스템 콜 함수 구현, 전역 파일 lock 추가(`struct lock filesys_lock`)

userprog/syscall.h

구현한 시스템콜 함수들에 대해 선언 추가

3. Synchronization in Filesystem

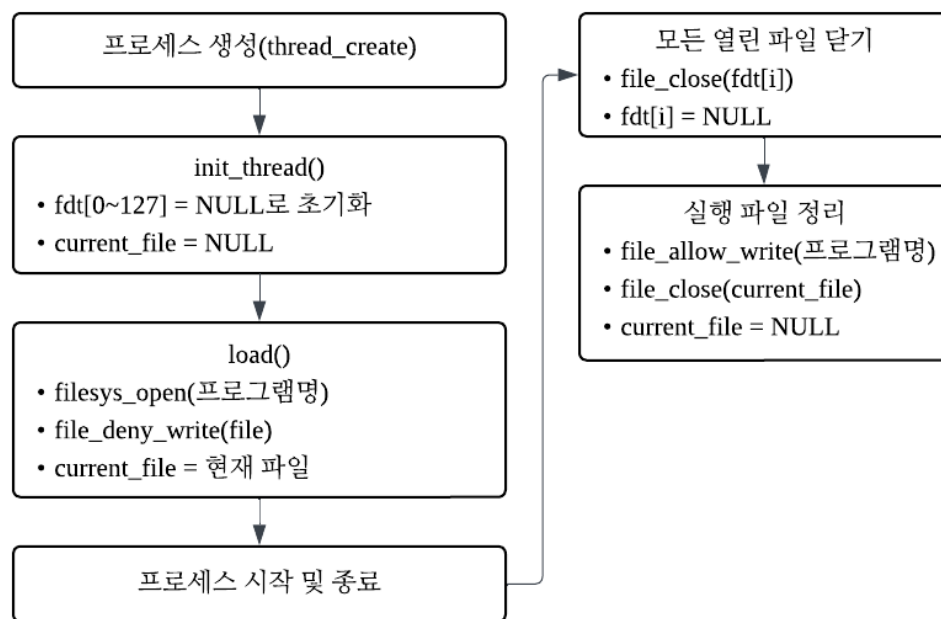
userprog/syscall.c

파일 시스템에 접근하는 시스템 콜 일부 함수들에 대해 락 획득 및 락 해지 추가

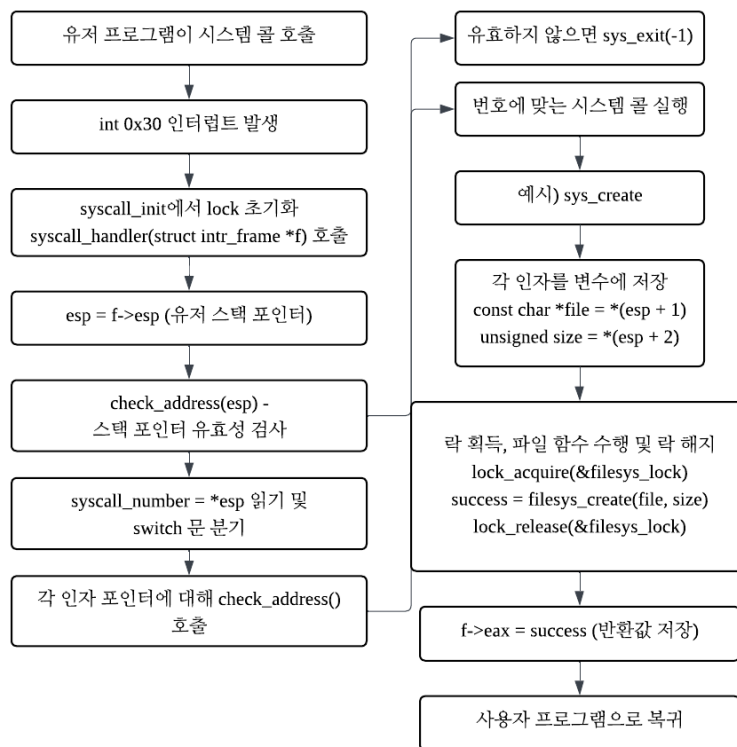
IV. 연구 결과

A. Flow Chart

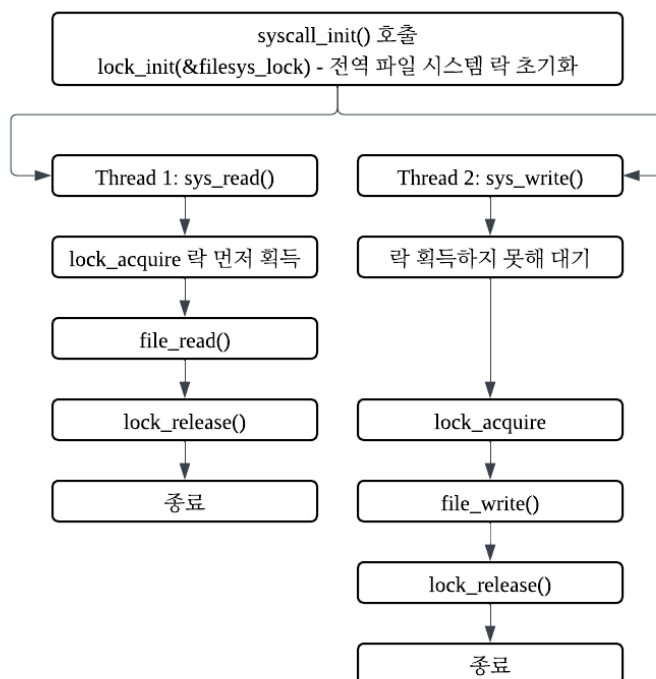
1. File Descriptor



2. System Calls



3. Synchronization



B. 제작 내용

- II. B. 개발 내용의 각 3가지 항목에 대하여 실질적으로 구현한 코드의 관점에서 작성 (구현 내용, 알고리즘 등을 명확히 서술할 것)

1. File Descriptor

File Descriptor Table 선언

```
#ifdef USERPROG
    struct file *fdt[128];
    struct file *current_file;
#endif
```

fdt는 개별 파일을 인식하기 위한 파일 디스크립터 테이블입니다. 0, 1, 2번 은 예약되어 있고 3번부터 할당될 예정입니다. current_file 포인터는 현재 실행 중인 파일에 대해 다른 프로세스가 접근해서 쓰는 것을 방지하기 위해 선언한 것으로 현재 실행 중인 파일의 포인터를 저장하는 변수입니다. 프로세스 시작 시 file_deny_write()를 이에 대해 호출하며 다른 프로세스의 쓰기를 방지하고 종료 시 file_allow_write()를 통해 이를 해제합니다.

프로세스 시작 시 File Descriptor Table 초기화 관련 구현: thread.c

```
static void init_thread(...) {
    ....
    for (int i = 0 ; i < 128; i++) {
        t->fdt[i] = NULL;
    }
    t->current_file = NULL;
}
```

Init_thread()함수에서 모든 entry를 NULL로 설정해줍니다.

프로세스 시작 시 실행 파일 write 금지 및 포인터 저장: process.c

```
static bool load(...) {
    struct thread *t = thread_current();
    ...
    file_deny_write(file);
    t->current_file = file;
}
```

load함수에서 현재 실행 파일에 대한 쓰기 금지 및 프로세스 종료 시 금지 해제를 위해 thread 구조체의 current_file 포인터에 현재 실행 파일의 포인터를 저장합니다.

프로세스 종료 시 file descriptor table 정리 및 write 금지 해제: process.c

```
void process_exit(void) {
    struct thread *cur = thread_current();
    for (int i = 3; i < 128; i++) {
        if (cur->fdt[i] != NULL) {
            file_close(cur->fdt[i]);
        }
    }

    if (cur->current_file != NULL) {
        file_allow_write(cur->current_file);
        file_close(cur->current_file);
    }
    ...
}
```

이전에 열려있는 파일 디스크립터 테이블에 대해서 열린 모든 파일들을 닫고 실행 파일에 대해 write 금지를 해제하며 종료합니다.

기존에는 next_fd를 사용하려고 했으나 재사용하려고 할 경우 비효율적일 거 같아, 매번 순회하는 방식으로 변경했습니다.

2. System Calls

```
case SYS_CREATE:
    check_address(esp + 1);
    check_address(*(esp + 1));
    check_address(esp + 2);
    f->eax = sys_create((const char *)*(esp + 1), (unsigned)*(esp + 2));
    break;
case SYS_REMOVE:
    check_address(esp + 1);
    check_address(*(esp + 1));
    f->eax = sys_remove((const char *)*(esp + 1));
    break;
case SYS_OPEN:
    check_address(esp + 1);
    check_address(*(esp + 1));
    f->eax = sys_open((const char *)*(esp + 1));
    break;
case SYS_CLOSE:
    check_address(esp + 1);
    sys_close((int)*(esp + 1));
    break;
case SYS_FILESIZE:
    check_address(esp + 1);
    f->eax = sys_filesize((int)*(esp + 1));
    break;
case SYS_SEEK:
    check_address(esp + 1);
    check_address(esp + 2);
    sys_seek((int)*(esp + 1), (unsigned)*(esp + 2));
    break;
case SYS_TELL:
    check_address(esp + 1);
    f->eax = sys_tell((int)*(esp + 1));
    break;
```

기존의 system call handler에 파일 시스템 관련 case를 추가하고 포인터 주소 검사도 하였습니다.

각 시스템 콜 함수 구현

sys_create() 구현

```
bool sys_create(const char *file, unsigned initial_size) {
    lock_acquire(&filesys_lock);
    bool success = filesys_create(file, initial_size);
    lock_release(&filesys_lock);
    return success;
}
```

filesys/filesys.c에 정의되어 있는 filesys_create() 함수를 사용하였습니다. 이 함수는 인자로 file명과 사이즈를 받습니다. 파일에 접근하기에 해당 함수 호출 위 아래로 lock을 획득하고 해제합니다. 그리고, 해당 함수가 반환한 값을 저장해 반환하며 종료합니다.

sys_remove() 구현

```
bool sys_remove(const char *file) {
    lock_acquire(&filesys_lock);
    bool success = filesys_remove(file);
    lock_release(&filesys_lock);
    return success;
}
```

filesys/filesys.c에 정의되어 있는 filesys_remove() 함수를 호출하고 위 아래로 lock을 획득하고 해제합니다. 해당 함수의 성공여부를 반환하며 종료합니다.

sys_open() 구현

```
int sys_open(const char *file) {
    struct thread *cur = thread_current();

    lock_acquire(&filesys_lock);
    struct file *f = filesys_open(file);
    lock_release(&filesys_lock);

    if (f == NULL) return -1;

    for (int fd = 3; fd < 128; fd++) {
        if (cur->fdt[fd] == NULL) {
            cur->fdt[fd] = f;
            return fd;
        }
    }

    lock_acquire(&filesys_lock);
    file_close(f);
    lock_release(&filesys_lock);
    return -1;
}
```

file 이름을 받아 lock을 얻고 filesys/filesys.c에 정의되어 있는 filesys_open 함수를 호출합니다. 만약 해당 열린 파일이 null이라면 -1을 반환하며 종료합니다. 그리고 file descriptor table을 돌며 가능한 가장 작은 배열 인덱스에 해당 포인터를 할당합니다. 그리고 fd를 반환하며 종료합니다. 그리고 만약 할당을 못했다면 lock을 획득하고 file_close() 함수를 사

용하여 해당 file을 닫습니다.

sys_close() 구현

```
void sys_close(int fd) {
    struct thread *cur = thread_current();
    if (fd < 3 || fd >= 128 || cur->fdt[fd] == NULL) return;
    lock_acquire(&filesys_lock);
    file_close(cur->fdt[fd]);
    lock_release(&filesys_lock);
    cur->fdt[fd] = NULL;
}
```

우선 인자로 받은 fd에 대하여 유효성을 확인합니다. 3 미만 128 이상이거나 해당 파일 테이블을 확인했을 때 NULL일 경우 -1을 반환하며 종료해줍니다. lock을 얻어 filesys/file.c에 정의되어 있는 file_close()를 호출하고 lock을 다시 해제합니다. 그리고 해당 인덱스의 fdt가 이상한 주소를 가리키지 않도록 NULL값을 넣습니다. 반환값은 따로 없습니다.

sys_filesize() 구현

```
int sys_filesize(int fd) {
    struct thread *cur = thread_current();
    if (fd < 3 || fd >= 128 || cur->fdt[fd] == NULL) return -1;
    lock_acquire(&filesys_lock);
    int size = file_length(cur->fdt[fd]);
    lock_release(&filesys_lock);
    return size;
}
```

우선 인자로 받은 fd에 대하여 유효성을 확인합니다. 3 미만 128 이상이거나 해당 파일 테이블을 확인했을 때 NULL일 경우 -1을 반환하며 종료해줍니다. 락을 선언하고 filesys/file.c에 정의되어 있는 file_length()함수를 호출하고 락을 해제하고 파일 사이즈 값을 반환하며 종료합니다.

sys_seek()

```
void sys_seek(int fd, unsigned position) {
    struct thread *cur = thread_current();
    if (fd < 3 || fd >= 128 || cur->fdt[fd] == NULL) return;
    file_seek(cur->fdt[fd], position);
}
```

우선 인자로 받은 fd에 대하여 유효성을 확인합니다. 3 미만 128 이상이거나 해당 파일 테이블을 확인했을 때 NULL일 경우 -1을 반환하며 종료해줍니다. 그리고 filesys/file.c에 정의되어 있는 file_seek()함수를 호출하는데 이 경우 lock이 필요 없습니다. 각 프로세스

의 file 구조체 내부의 pos 필드에 접근하기 때문입니다. 반면 read나 write 같은 다른 file 함수의 경우 여러 프로세스가 공유하고 접근하는 inode 같은 파일 시스템 자료구조에 접근해야 하기 때문에 필요합니다. 반환값은 따로 존재하지 않습니다.

sys_tell() 구현

```
unsigned sys_tell(int fd) {  
    struct thread *cur = thread_current();  
    if (fd < 3 || fd >= 128 || cur->fdt[fd] == NULL) return -1;  
    return file_tell(cur->fdt[fd]);  
}
```

우선 인자로 받은 fd에 대하여 유효성을 확인합니다. 3 미만 128 이상이거나 해당 파일 테이블을 확인했을 때 NULL일 경우 -1을 반환하며 종료해줍니다. 그리고 filesys/file.c에 정의되어 있는 sys_tell()함수를 호출하여 offset 즉 위치를 반환합니다. 이 경우 락이 불필요한데 seek과 같이 각 프로세스의 file 구조체 내부의 pos 필드에 접근하기 때문입니다.

sys_read() 수정

```
int sys_read(int fd, void *buffer, unsigned size) {  
    unsigned cnt = 0;  
    char *buf = (char *)buffer;  
    struct thread *cur = thread_current();  
    // fd가 0인 경우 (표준 입력) ==> input_getc() 사용  
    if (fd == 0) {  
        while (cnt < size) {  
            char c = input_getc();  
            buf[cnt++] = c;  
            // if (c == '\n') break;  
        }  
        // buf[cnt] = '\0';  
        return cnt;  
    }  
    if (fd < 3 || fd >= 128 || cur->fdt[fd] == NULL) return -1;  
  
    lock_acquire(&filesys_lock);  
    int bytes = file_read(cur->fdt[fd], buffer, size);  
    lock_release(&filesys_lock);  
    return bytes;  
}
```

Fd가 0인 경우에 대해서는 기존의 프로젝트 1과 같고, 이외의 경우에 대해 추가적으로 구현했습니다. 인자로 받은 fd에 대하여 유효성을 확인합니다. 3 미만 128 이상이거나 해당 파일 테이블을 확인했을 때 NULL일 경우 -1을 반환하며 종료해줍니다. 락을 획득하고 filesys/file.c에 정의되어 있는 file_read()함수를 호출하고 락을 해제하고 읽은 bytes수를 반환하며 종료합니다.

sys_write()수정

```
int sys_write(int fd, const void *buffer, unsigned size) {
    // check_address(buffer);
    struct thread *cur = thread_current();
    if (fd == 1) {
        // printf("SYS_WRITE buffer content: %.*s\n", size, (char *)buffer);
        putbuf(buffer, size);
        return size;
    }
    if (fd < 3 || fd >= 128 || cur->fdt[fd] == NULL) return -1;
    lock_acquire(&filesys_lock);
    int bytes = file_write(cur->fdt[fd], buffer, size);
    lock_release(&filesys_lock);
    return bytes;
}
```

인자로 받은 fd가 1인 경우 기존의 프로젝트 1과 같습니다. 이 외의 경우에 대해 추가적으로 구현하였습니다. 우선 인자로 받은 fd에 대하여 유효성을 확인합니다. 3 미만 128 이상이거나 해당 파일 테이블을 확인했을 때 NULL일 경우 -1을 반환하며 종료해줍니다. 그리고 락을 획득하고 filesys/file.c에 정의되어 있는 file_write() 함수를 실행하고 락을 해제하고, 쓴 bytes 수를 반환합니다.

3. Synchronization

syscall.c에 struct lock filesys_lock(파일 시스템 락)을 전역 변수로 선언했습니다. 그리고 이를 syscall_init() 함수에서 초기화했습니다.

```
void syscall_init(void) {
    intr_register_int(0x30, 3, INTR_ON, syscall_handler, "syscall");
    lock_init(&filesys_lock);
}
```

그리고 파일 관련 시스템 콜 함수에서 획득하도록 했습니다. 효율성 개선을 위해 해당 시스템 콜들에서 락이 필요 없는 local 작업에 대해서는 lock 밖에서 수행되도록 하였습니다. 이러한 코드에 대해서는 2번 항목과 같기에 생략하겠습니다. 가장 중점적으로 개선한 부분은 앞서 말했듯이 File_seek()과 file_tell()은 pos 필드만 읽고 쓰기에 락 없이도 안전하다고 판단하여 이를 쓰지 않았습니다.

추가 구현: page_fault 수정

```
static void
page_fault (struct intr_frame *f)
{
    bool not_present; /* True: not-present page, false: writing r/o page. */
    bool write;        /* True: access was write, false: access was read. */
    bool user;         /* True: access by user, false: access by kernel. */
    void *fault_addr;  /* Fault address. */

    /* Obtain faulting address, the virtual address that was
       accessed to cause the fault. It may point to code or to
       data. It is not necessarily the address of the instruction
       that caused the fault (that's f->eip).
       See [IA32-v2a] "MOV---Move to/from Control Registers" and
       [IA32-v3a] 5.15 "Interrupt 14---Page Fault Exception
       (#PF)". */
    asm ("movl %%cr2, %0" : "=r" (fault_addr));

    /* Turn interrupts back on (they were only off so that we could
       be assured of reading CR2 before it changed). */
    intr_enable ();

    /* Count page faults. */
    page_fault_cnt++;

    /* Determine cause. */
    not_present = (f->error_code & PF_P) == 0;
    write = (f->error_code & PF_W) != 0;
    user = (f->error_code & PF_U) != 0;

    if (user) sys_exit(-1);
    /* To implement virtual memory, delete the rest of the function
       body, and replace it with code that brings in the page to
       which fault_addr refers. */
    printf ("Page fault at %p: %s error %s page in %s context.\n",
            fault_addr,
            not_present ? "not present" : "rights violation",
            write ? "writing" : "reading",
            user ? "user" : "kernel");
    kill (f);
}
```

기존에는 Page_fault 발생시 커널 패닉이 일어나 핀토스 시스템 자체가 중단되었지만, user 모드에서 page fault 발생시 sys_exit(-1)을 호출하며 해당 프로세스만 종료되도록 수정했습니다.

C. 시험 및 평가 내용

```
pass tests/userprog/args-none
pass tests/userprog/args-single
pass tests/userprog/args-multiple
pass tests/userprog/args-many
pass tests/userprog/args-dbl-space
pass tests/userprog/sc-bad-sp
pass tests/userprog/sc-bad-arg
pass tests/userprog/sc-boundary
pass tests/userprog/sc-boundary-2
FAIL tests/userprog/sc-boundary-3
pass tests/userprog/halt
pass tests/userprog/exit
pass tests/userprog/create-normal
pass tests/userprog/create-empty
pass tests/userprog/create-null
pass tests/userprog/create-bad-ptr
pass tests/userprog/create-long
pass tests/userprog/create-exists
pass tests/userprog/create-bound
pass tests/userprog/open-normal
pass tests/userprog/open-missing
pass tests/userprog/open-boundary
pass tests/userprog/open-empty
pass tests/userprog/open-null
pass tests/userprog/open-bad-ptr
pass tests/userprog/open-twice
pass tests/userprog/close-normal
pass tests/userprog/close-twice
pass tests/userprog/close-stdin
pass tests/userprog/close-stdout
pass tests/userprog/close-bad-fd
pass tests/userprog/read-normal
pass tests/userprog/read-bad-ptr
pass tests/userprog/read-boundary
pass tests/userprog/read-zero
pass tests/userprog/read-stdout
```

```
pass tests/userprog/read-bad-fd
pass tests/userprog/write-normal
pass tests/userprog/write-bad-ptr
pass tests/userprog/write-boundary
pass tests/userprog/write-zero
pass tests/userprog/write-stdin
pass tests/userprog/write-bad-fd
pass tests/userprog/exec-once
pass tests/userprog/exec-arg
pass tests/userprog/exec-bound
FAIL tests/userprog/exec-bound-2
FAIL tests/userprog/exec-bound-3
pass tests/userprog/exec-multiple
pass tests/userprog/exec-missing
pass tests/userprog/exec-bad-ptr
pass tests/userprog/wait-simple
pass tests/userprog/wait-twice
pass tests/userprog/wait-killed
pass tests/userprog/wait-bad-pid
pass tests/userprog/multi-recurse
pass tests/userprog/multi-child-fd
pass tests/userprog/rox-simple
pass tests/userprog/rox-child
pass tests/userprog/rox-multichild
pass tests/userprog/bad-read
pass tests/userprog/bad-write
pass tests/userprog/bad-read2
pass tests/userprog/bad-write2
pass tests/userprog/bad-jump
pass tests/userprog/bad-jump2
pass tests/userprog/no-vm/multi-oom
pass tests/filesys/base/lg-create
pass tests/filesys/base/lg-full
pass tests/filesys/base/lg-random
pass tests/filesys/base/lg-seq-block
pass tests/filesys/base/lg-seq-random
```

```
pass tests/filesys/base/sm-create
pass tests/filesys/base/sm-full
pass tests/filesys/base/sm-random
pass tests/filesys/base/sm-seq-block
pass tests/filesys/base/sm-seq-random
pass tests/filesys/base/syn-read
pass tests/filesys/base/syn-remove
pass tests/filesys/base/syn-write
```

- 채점범위가 아닌 FAIL tests/userprog/sc-boundary-3, FAIL tests/userprog/exec-bound-2, FAIL tests/userprog/exec-bound-3 3개의 케이스 말고는 다 성공했습니다.